Week 3 Project

Build Your AI Agent

Submission Link: https://forms.gle/HMgTU7zy6UJ8XkJX6

The submission deadline is **Sunday, August 30, 2026** to be considered for Builder of the Week. Completing this project is also mandatory for The Gen Academy certification, with the final certification deadline being **September 16, 2026**.

If you have any questions, please email tanish@thegenacademy.com.

What this week is about

This week you are building an agentic system. Not a one-shot LLM call. Not a RAG lookup. An agent that decides what to do next, calls tools, holds state across steps, recovers from errors, and hands off to a human when it should. By Friday you will have it running end-to-end on a real task.

The hard parts of agentic systems are not the prompts. They are control flow, state, tool failure, and the boundary between what the agent does autonomously and what needs a human. The framework on page 3 forces a decision on each of those.

How this project works

You will make two independent choices.

Choice	Option 1	Option 2
Step 1: Pick your use-case	Pick one of six suggested use cases (page 5). Each comes pre-scoped using the framework.	Bring your own use case.
Step 2: Pick your build track	No-code with n8n/Lyzr. Visual workflow builder with AI Agent, tool, and trigger nodes.	Code-heavy with LangChain + LangGraph. Stateful graph flows in Python, vibe-codeable with Codex.

Part 1: Pick a use-case

Each of the six use cases below is a fully filled framework. Pick one, adapt the user and tools to your context, and you have a Week 3 scope.

For each of the six agents, you can choose whatever is the most appropriate agent pattern (single ReAct, multi-agent, pipeline, voice).

PS: We have intentionally NOT added minute details in the 6 use cases skills below because we would like you to be creative and think about what each use case agent pattern should look like.

#	Use case
A	Market Research Agent: Competitor Analysis
B	Multi-Agent Deal Review Pipeline
C	Intelligent Project Status Agent
D	GTM Agent: Ideation to Copy
E	Code Review Agent
F	ElevenLabs Voice Agent: ITSM

> **Want to bring your own?**
>
> If you would like to build something which is not one of the use cases mentioned above, we would highly recommend picking one of the common use cases for which you can find existing resources on any ten docs or LangChain docs. It's fairly easy for you to build it end to end.

Whether you have picked one of the six use cases that we have suggested or you have an own use case in mind, we would highly recommend that you formalize it using the agent framework below.

Project 3A: Market Research Agent — Competitor Analysis

Problem Statement

Design a multi-agent market research system for competitor analysis:

- Agent 1 discovers a company's top 3 competitors using the you.com Search API
- Agent 2 gathers fresh web and news data for each competitor
- Agent 3 extracts pricing, core features, market positioning, and recent news
- and an Orchestrator Agent coordinates the full research pipeline and compiles the findings into structured competitor analysis briefings.

Add other subagents as needed. Modeled on real competitive intelligence workflows used by product teams, strategy teams, consultants, and founders. This project demonstrates autonomous

web research, tool use, delegation, stateful orchestration, and structured analysis. Build with LangChain/LangGraph or another framework of your choice; low-code builders can implement an equivalent workflow in n8n. Use you.com as the primary search source and present the results through a simple interface such as Streamlit.

Best For

PMs (competitive intelligence), Consultants (client research), Executives (market monitoring), Analysts (industry tracking), Founders (landscape analysis), Students (learning research workflows).

Deliverable

Working research agent that produces formatted competitor analysis briefings + sample output for 2–3 competitors.

Submission

Demo recording + GitHub link or zip file.

Difficulty

Beginner to Intermediate | Low-code track (n8n) and code track (LangChain/LangGraph) available

Project 3B: Multi-Agent Deal Review Pipeline

Description

Design a multi-agent system for financial deal review:

- Agent 1 extracts key terms from a deal document
- Agent 2 checks them against compliance rules
- Agent 3 flags risks and generates a summary
- and an Orchestrator Agent coordinates the full pipeline.

Add other subagents as needed. Modeled on real financial services workflows — loan approvals, insurance underwriting, investment reviews. This project demonstrates delegation, coordination, and agent-to-agent communication at scale. Build with LangChain/LangGraph or another framework of your choice.

Best For

Financial Services roles (JPMC, Wells Fargo, Citi, Morgan Stanley, NYLife), Architects (pipeline design), Consultants (process automation), Program Managers (workflow optimization).

Deliverable

Working multi-agent pipeline with 3+ specialized agents, orchestration logic, and a sample deal review output.

Submission

Demo recording + GitHub link

Difficulty

Advanced | Code track uses LangChain/LangGraph | Business track designs the workflow and tests outputs using n8n

Project 3C: Intelligent Project Status Agent

Description

Build an agent that connects to your project management tools (Jira, Asana, Notion), pulls current sprint status, identifies blockers, and generates a weekly status report with risk flags. Add memory so the agent tracks week-over-week trends and can answer questions like “What’s been stuck for more than one sprint?” For the code track, consider using mem0 as the persistent memory layer so the agent can store project updates across sources and retrieve the right historical context for status questions. Low-code builders use n8n to wire up the integrations; engineers build with LangChain/LangGraph or your preferred framework. This is the agent every program manager wishes they had — and every engineering lead would actually use.

Best For

Program/Project Managers (direct workflow value), Tech Leads (sprint management), Engineering Managers, Consultants (client project tracking), Delivery Managers.

Deliverable

Working status agent, memory, and a sample weekly status report.

Submission

Demo recording + GitHub link

Difficulty

Intermediate | Low-code track (n8n) and code track (LangChain/LangGraph) available

Project 3D: GTM Agent — Ideation to Copy

Description

Build an agent that takes a product, feature, or upcoming event as input and autonomously generates a full go-to-market content suite: a LinkedIn post, a promotional email, a short blog draft, and ad copy variations. The agent ingests a calendar of events or a list of products document (PDF, Google Sheet, or Notion export) into a vector store, then uses RAG to pull relevant context — launch dates, event details, product specs, past campaign messaging — before generating content. It researches the product, identifies the target audience, selects a tone, and produces ready-to-edit content across formats. A review agent critiques and improves each piece, checking for consistency across formats, tone alignment, and factual grounding against the source documents.

Best For

PMs (product launches), Marketing-adjacent roles, Founders (content for their startup), Consultants (client-facing content), Executives (thought leadership pipeline), Students (portfolio content).

Deliverable

Working GTM content agent that produces multi-format output + sample content suite for one product/feature.

Submission

Demo recording + GitHub link

Difficulty

Beginner to Intermediate | Low-code track (n8n) and code track (LangChain/LangGraph) available

Project 3E: Code Review Agent

Description

Build a multi-agent code review system: Agent 1 analyzes code for bugs and anti-patterns, Agent 2 checks for security vulnerabilities, Agent 3 evaluates test coverage and suggests missing tests. An orchestrator combines findings into a prioritized review report with severity ratings. Test on real pull requests or open-source repos. Build with LangChain/LangGraph or your preferred framework. This is directly applicable to engineering workflows at companies shipping code at scale.

Best For

Software Engineers (Atlassian, Razorpay, Cisco, JPMC), DevSecOps Engineers (Workday), QA/SDET (Equifax, Affinity), AI Engineers (Google, Ericsson), Tech Leads.

Deliverable

Multi-agent code review system tested on 3 real code samples + a prioritized findings report.

Submission

Demo recording + GitHub link

Difficulty

Advanced | Code required | PMs can evaluate outputs and design the review rubric

Project 3F: Multi-Agent IT Support Voice Agent

Description

Build a voice-based AI support system that can handle employee IT support calls from intake to resolution.

The system should be able to understand the caller's issue, verify basic employee details, route the request to the right support path, attempt to resolve the problem, and escalate when needed. After the call, the system should generate a structured review of what happened, including whether the issue was resolved, whether the right process was followed, and whether any follow-up is required.

This project demonstrates how multiple specialized AI agents can work together in a real support workflow. Instead of one general-purpose agent handling everything, the system uses different agents for intake, issue handling, escalation, and quality review.

Learners can build this using ElevenLabs Conversational AI. Advanced learners can extend the system with tools, knowledge base search, ticket creation, or custom backend APIs.

Best For

IT Support teams, Support Operations, Voice AI builders, Solutions Architects, Program Managers, QA/Compliance teams, AI Engineers, and students interested in real-world multi-agent workflow design.

Submission

Demo recording + Project documentation

Difficulty

Intermediate

Part 2: The Agent Framework

The Primer: Your one-liner

Before the framework, write a single sentence that captures the whole agent. If you cannot say it in one line, you have not yet decided what your agent does autonomously, what it hands off, or how you will know it worked.

> My agent helps **[USER]** do **[MULTI-STEP TASK]** in **[SURFACE]**, replacing **[the manual workflow they use today and what it costs them]**. It does **[the work]** on its own using **[N tools]**, hands off to a human **[when ___]**, and I'll know it works when **[USER]** can **[complete the task]** in under **[TIME]** with **[a clear success rate]**.

Worked example

My agent helps a startup founder research a new market in a web app, replacing the 6 hours of manual googling and tab-juggling it takes to size up competitors today. It searches the web, pulls each competitor's pricing and positioning, and drafts a one-page brief on its own using 4 tools, hands off to the founder to review before the brief gets saved, and I'll know it works when a founder can get a usable market brief in under 15 minutes that they'd actually send to their team 8 times out of 10.

> **Three rules for the one-liner**
>
> **Task completion, not single-shot accuracy.** Your agent succeeds when the user finishes a workflow, not when one model call returns a good answer. Measure end-to-end.
>
> **State is the hard part.** Decide what your agent remembers, for how long, and where it lives (session memory, conversation history, persistent store). Hand-waving here breaks the demo.
>
> **Write actions deserve a human.** Any action that creates, modifies, sends, or pays should default to human approval. Reads can be autonomous. Be deliberate about which is which.

The Framework (optional if you want to be very detailed)

Fill out every field in 1 to 2 sentences. The framework forces a decision at every layer of the agent so nothing gets hand-waved.

Field	Fill in (1 to 2 sentences max)
Agent goal (one line)	The one job, in a sentence. "Takes a customer email and drafts a reply.
Where do people use it?	Slack, web chat, IDE, voice, email, internal portal.
What steps does it take, in order?	List them 1, 2, 3. (This is control flow, without the word.)
What can it actually do?	List 3 to 6 actions or tools. Mark which ones just look things up, and which ones *change* something (send, post, update, delete).
What does it need to remember?	Just this conversation, or things across sessions? A name, an order number, past messages?
What should it never do?	The hard limits. Never send money, never delete records, never share personal info.
Human-in-the-loop	Where humans review (after plan / before write / final approval), and how they intervene.
What happens when something breaks?	A tool returns nothing or errors out. Does it retry, ask the user, or stop?
How do you know it worked?	One clear measure. "Drafted a usable reply 8 times out of 10."

Part 3: Pick Your Build Track

The framework is identical for both tracks. The track determines how you implement it.

Track 1: No-code with (n8n or Lyzr)

What it is
Visual, low-code agent builders that let you assemble AI workflows using nodes/components for agents, tools, memory, triggers, and control flow with minimal or no Python required.

Best for
Rapid prototyping, integrations-heavy agents (CRM, helpdesk, Slack, calendar), demos for non-technical stakeholders, and agents that primarily orchestrate APIs and existing services.

Key building blocks
AI Agent components, tool/API integrations, memory, webhook or event triggers, conditional logic, workflow controls, and human-in-the-loop steps.

Tradeoffs
You gain speed and accessibility at the cost of some control. Complex multi-agent architectures, sophisticated state machines, custom retry logic, and highly specialized orchestration can become harder to implement or maintain. The tradeoff is generally depth and flexibility for faster development.

Note: Lyzr credits have been provided.

Track 2: Code-heavy with LangChain + LangGraph

What it is
Python framework with agent and tool primitives (LangChain) plus LangGraph for stateful, multi-step graph flows. Pair with Codex or Claude Code for vibe-coding speed.

Best for
Multi-agent systems, complex state, custom planning, real-time voice (ElevenLabs SDK), production-grade systems, anyone writing evals as code.

Key building blocks
LangChain agents and tools, LangGraph state machines, checkpointers for persistence, interrupts for human-in-the-loop, LangSmith for tracing and evals.

Tradeoffs
Steeper ramp, more code to maintain. The upside is full control and a real portfolio piece. Codex vibe-coding closes the speed gap considerably.

How to decide

Default to Track 2 (LangChain + LangGraph) if your agent is multi-agent, voice-based, or has complex state. n8n/Lyzr cannot easily express these.

Pick Track 1 if your agent mostly orchestrates SaaS APIs, you need a demo by Wednesday, or you do not write Python. You can always rebuild it in code later.

Both tracks may use Nebius Token Factory for at least one model call, so we can compare different approaches during the cohort review. Fireworks access will also be provided for

participants who prefer to use it. Both tracks are free to use Codex, Claude Code, or Cursor to accelerate development, especially for Track 2.

How to submit

Deliverables for Week 3

Project documentation	Submit a Google Doc explaining what you built. Include: project overview, datasets used, prompts you used during vibe coding, iterations you tried, and any learnings or observations from the workflow.
Video demo	Submit a video (5 minutes or less) where you walk through your application, explain what you built, describe how you used AI coding tools, and demonstrate the final result live.
Code base	Upload your code assets to Github and share a link in the form below

One last thing

If your agent works on the happy path but falls over on the first tool failure, you have not finished. Spend the last day on error handling and human-in-the-loop. That is what separates a demo from a build.

If you get stuck, post your one-liner, your architecture diagram, and the specific failure you are hitting. Specific questions get specific help.

** Solutions for use-cases

Given below are some solutions that we have put together for the use cases shared in this particular week's suggested use case section (part 1).

We highly encourage you **NOT** to look at this before you get started with your project. We intentionally don't want you to go through this because it will direct your thinking in a particular direction.

We would rather want you to think through the solution and build this yourself, even if it takes you a little bit more time. Only refer to the following documents if you are absolutely stuck and

are unable to make progress. Use them as a hint document rather than replicating the following solutions. If you end up replicating the following solutions, you will not be given scores.

#	Use case	Code Track	No-code Track
1	Market Research Agent: Competitor Analysis	Langchain and Lan…	n8n
2	Multi-Agent Deal Review Pipeline	Langchain and Lan…	n8n
3	Intelligent Project Status Agent	Langchain and Lan…	n8n
4	GTM Agent: Ideation to Copy	LangChain and Lan…	n8n
5	Code Review Agent	Langchain and Lan…	n8n
6	ElevenLabs Voice Agent: ITSM		IT Support Voice A…

** Lyzr Solutions

Built on Lyzr Studio's visual SuperFlow builder. The same rule applies as above: treat these as hint documents, not templates. Replicating them scores zero.

#	Use case	Lyzr Solution
1	Multi-Agent Deal Review Pipeline	Lyzr